# PRD — Frontend Shell: Login & Dashboard

## Textile/Yarn ERP System (Next.js)

**Document status:** Draft v0.1 **Scope:** Frontend only — login screen and dashboard shell **Decided so far:** Frontend framework = Next.js. Everything else (backend, database, hosting, auth provider, final module list) is **open** and this PRD is written so none of those decisions are locked in by it.

---

## 1. Purpose

Before backend, database, or module scope for the textile/yarn ERP are finalized, we want to start building the **frontend shell** — the login flow and the dashboard layout that every module (Finance, Inventory, Weaving, HR, etc.) will eventually plug into.

This PRD defines that shell as an **independent, backend-agnostic** piece: it should work against a mocked/stubbed data layer today, and swap to a real API later (REST, GraphQL, tRPC, Supabase, Firebase, .NET, Node, whatever gets picked) without a rewrite.

## 2. Goals

- Ship a working login screen and dashboard shell in Next.js that doesn't assume any specific backend, auth provider, or database.
- Establish the navigation/module framework the eventual ERP modules (GL, AP/AR, Inventory, Weaving, HRMS/Payroll) will slot into later.
- Keep every integration point (auth, data fetching, user/role model) behind an interface/adapter so the real decisions can be made later without touching UI code.
- Support role-based views from day one at the UI level (even before real roles/permissions exist on a backend), since the source proposal implies distinct modules per department (Finance, Inventory/Store/Gate, Weaving, HR/Payroll).

## 3. Non-Goals (explicitly out of scope for this PRD)

- Choosing the backend framework, language, or hosting.
- Choosing the database engine.
- Choosing the auth provider/strategy (JWT vs sessions vs OAuth vs SSO) — the login screen will target a **swappable auth interface**, not a specific implementation.
- Building any actual ERP module logic (GL, Inventory, Weaving contracts, Payroll, etc.) — this PRD only covers the shell they will live inside.
- Mobile app — noted as a future requirement, not addressed here, but layout choices should not preclude it (e.g., avoid desktop-only patterns where reasonably avoidable).

## 4. Guiding Principle: Adapter Pattern for Everything Uncertain

Since nothing but "Next.js frontend" is confirmed, the design should isolate uncertainty:

| Layer | Status | How it's handled here |
|---|---|---|
| UI components (login form, dashboard shell, nav) | Can be built now | Built directly in Next.js |
| Auth mechanism | Undecided | Behind an `AuthProvider` interface (e.g., `login()`, `logout()`, `getSession()`) with a mock implementation for now |
| Data fetching | Undecided | Behind a data-access layer (e.g., a thin API client module) so REST/GraphQL/tRPC/BFF can be swapped later |
| User/role model | Undecided | A simple typed shape (`User { id, name, role, modules[] }`) used by the UI; real shape can be reconciled later |
| Styling/design system | Open | Recommend Tailwind + a component library (shadcn/ui) — flexible, easy to reskin later, not a backend dependency |

## 5. User Roles (assumed, not final)

Based on the modules referenced in the original ERP proposal (Financial Accounting, Inventory/Store/Gate, Weaving, HR/Payroll), the UI should assume **multiple role types exist** without hardcoding what they are:

- Admin / Super User
- Finance user (GL/AP/AR)
- Inventory/Store/Gate user
- Weaving/Production user
- HR/Payroll user

The dashboard should render **modules based on the logged-in user's assigned modules/roles**, driven by a config/data object — not by separate hardcoded pages per role. This keeps it flexible if the final module list or role structure changes.

## 6. Functional Requirements

### 6.1 Login Screen

- Fields: username/email + password (matches typical on-prem ERP patterns like the original Oracle Forms system).
- Client-side validation (required fields, basic format check).
- Loading and error states (invalid credentials, network/server error) — driven by the mock `AuthProvider` for now.
- "Remember me" toggle (UI only for now; persistence mechanism TBD with auth decision).
- Placeholder hooks for future needs without committing to them now: forgot-password link, multi-factor step, SSO button — visually present/stubbed if desired, but not functional yet.
- On success: redirect to `/dashboard`.
- Session/route protection: unauthenticated users hitting `/dashboard` (or any module route) redirect to `/login`. Implemented via a route-guard wrapper that reads from the mock auth/session layer, so it can be pointed at a real session check later.

### 6.2 Dashboard Shell

- **Layout:** persistent sidebar (module/nav list) + top bar (user info, logout, org/company name) + main content area.
- **Sidebar navigation:** driven by a config list of modules (e.g., Finance, Inventory, Weaving, HR) filtered by the current user's assigned modules/roles. Modules with no real page yet show a placeholder ("Coming soon") page rather than breaking navigation.
- **Top bar:** logged-in user name/role, company/branch selector (stub, since multi-branch/multi-location is implied by the original proposal's mention of WAN/multi-location deployment), logout action.
- **Home/landing dashboard view:** a widget-style summary area (cards/tiles) — can start with placeholder tiles (e.g., "Pending approvals", "Recent activity") wired to mock data, so real KPIs can be swapped in per-module later.
- **Empty/loading/error states** for all dashboard widgets, since real data sources aren't chosen yet.
- **Responsive layout:** sidebar collapses on smaller screens; keeps door open for the future mobile app without building it now.

### 6.3 Session & Logout

- Logout clears the mock session and redirects to `/login`.
- Session expiry handling stubbed (e.g., a `getSession()` call that can later return "expired" from a real backend).

## 7. Technical Approach (frontend-only)

- **Framework:** Next.js (App Router recommended for layout nesting — sidebar/topbar as a shared layout wrapping all authenticated routes).
- **Styling:** Tailwind CSS + shadcn/ui components (buttons, inputs, cards, sidebar primitives) — fast to build, easy to theme, doesn't lock in any backend choice.
- **State/session:** React context or a lightweight state library (e.g., Zustand) holding the current user/session object, populated by the mock `AuthProvider`.
- **Mock data layer:** a local module (e.g., `lib/auth-mock.ts`, `lib/api-mock.ts`) simulating login, session check, and dashboard widget data with realistic delays/error cases, so the UI is tested against real-ish conditions before any backend exists.
- **Routing structure (example, adjustable):**
  - `/login`
  - `/dashboard` (landing/home)
  - `/dashboard/finance`, `/dashboard/inventory`, `/dashboard/weaving`, `/dashboard/hr` — placeholder pages per module for now.

## 8. Success Criteria

- A user can "log in" (against the mock layer) and land on a dashboard whose sidebar reflects their assigned modules.
- Swapping the mock `AuthProvider`/data layer for a real backend later requires **no changes to page/component code** — only to the adapter implementations.
- Layout and navigation accommodate adding/removing modules (e.g., if Weaving is dropped or a new module is added) via config, not code restructuring.

## 9. Open Questions (deliberately unresolved)

- Backend framework/language and database engine.
- Auth strategy (custom, NextAuth/Auth.js, third-party IdP, SSO).
- Final role/permission model (flat roles vs. granular permissions per module/action).
- Multi-branch/multi-location handling at the data level (UI only stubs a selector for now).
- Whether "Option A" (license + monthly maintenance) vs "Option B" (pure subscription) style commercial model from the original proposal has any bearing on how the product is packaged/deployed (e.g., single-tenant per client vs. multi-tenant SaaS) — this affects auth/session design later but not this frontend shell.

---

*This document intentionally avoids backend, database, and auth-provider decisions so the login/dashboard frontend can be built now and re-pointed at real infrastructure once those choices are made.*